

CI/CD Pipeline Implementation using GitHub, Jenkins, Docker, and Kubernetes

-18 MAY 2026

NAME: SEETHA LAKSHMI K A P

EMAIL: seethalakshmikap@gmail.com

GITHUB REPO LINK:

<https://github.com/seethalakshmii/simple-node-app.git>

PROJECT DEMO:

<https://drive.google.com/file/d/1HHSab7Bj8-XspkBzwdGTeuukR2gGSOul/view?usp=sharing>

1. INTRODUCTION

1.1 Project Overview

This project implements a fully automated **DevSecOps CI/CD pipeline** that enables continuous integration, security validation, and continuous deployment of a containerized application using **Jenkins, Docker, GitHub, and Kubernetes**.

The system follows modern cloud-native practices where every code commit triggers an automated pipeline that builds, tests, scans, packages, and deploys the application across multiple environments.

1.2 Problem Statement

Manual software deployment processes are slow, error-prone, and inconsistent. This project solves this by implementing an automated pipeline that ensures:

- Faster delivery cycles
- Consistent deployments
- Secure container images
- Multi-environment isolation
- Automated rollback on failure

1.3 Objectives

- Automate CI/CD workflow using Jenkins pipeline (Groovy)
- Trigger pipeline using GitHub webhook via ngrok
- Build Docker images for consistent deployment
- Perform security scanning using Trivy (DevSecOps)
- Deploy to Kubernetes environments (Dev, Staging, Production)
- Enable rollback on deployment failure
- Ensure observability using logging and monitoring tools

1.4 Technology Stack

- Git & GitHub – Source control
 - Jenkins – CI/CD orchestration
 - Docker – Containerization
 - Kubernetes (Minikube) – Deployment platform
 - Trivy – Security scanning tool
 - ngrok – Webhook tunneling tool
 - Node.js – Application runtime
-

2. SYSTEM ARCHITECTURE

Developer → GitHub → ngrok Tunnel → Jenkins Pipeline → Build → Test → Docker Image → Trivy Scan → DockerHub → Kubernetes Cluster → Deployment

2.1 Workflow Explanation

1. Developer pushes code to GitHub repository
 2. GitHub triggers Jenkins using ngrok public URL
 3. Jenkins pulls latest code
 4. Application dependencies installed
 5. Unit tests executed
 6. Docker image is built
 7. Trivy scans image for vulnerabilities
 8. Image is pushed to DockerHub registry
 9. Kubernetes deploys application to environments
 10. Rollout status monitored automatically
-

3. IMPLEMENTATION DETAILS

3.1 Jenkins Setup

Installed Plugins:

- Git Plugin
- Pipeline Plugin
- Docker Pipeline Plugin
- Credentials Binding Plugin

3.2 Required Tools on Jenkins Server

- **Docker Engine** → Used for building and pushing container images
- **kubectl** → Used for deploying and managing applications in Kubernetes
- **Minikube** → Local Kubernetes cluster used for deployment testing
- **Node.js** → Required to run and test the application
- **Trivy (via Docker)** → Used for security scanning of Docker images
- **ngrok** → Used to expose Jenkins locally for GitHub webhook triggers

3.3 GitHub + ngrok Integration

Since Jenkins runs locally, it is not publicly accessible. Therefore, **ngrok** is used to expose the Jenkins server to the internet securely. This enables GitHub webhooks to trigger Jenkins builds automatically on every code push.

Setup Steps:

- **Step 1:** Start Jenkins - <http://localhost:8080>
- **Step 2:** Start ngrok tunnel - `ngrok http 8080`
- **Step 3:** Public URL generated by ngrok - <https://unsuited-smolder-rimmed.ngrok-free.dev>
- **Step 4:** Configure GitHub Webhook - Add the webhook URL in GitHub repository, <https://unsuited-smolder-rimmed.ngrok-free.dev/github-webhook/>

3.4 Kubernetes Setup

- **Start Cluster :** `minikube start`
- **Create Environments :**
 - `kubectl create namespace dev`
 - `kubectl create namespace staging`
 - `kubectl create namespace prod`
- **Deployment Strategy**
 - Dev → Feature testing
 - Staging → Pre-production validation
 - Production → Live deployment

Uses rolling updates to ensure zero downtime

3.5 DockerHub Setup

- Create repository: `seetha88/simple-node-app`
- Store credentials in Jenkins:
 - ID: `dockerhub-creds`

3.6 Jenkins Pipeline (Groovy Script)

Implements full CI/CD lifecycle:

- Checkout
- Build
- Test
- Security scan
- Push image
- Deploy
- Rollback

4. TESTING & VALIDATION

4.1 Unit Testing

- `npm test` is used to execute unit tests for the application.
- Tests are written using Jest, which ensures application logic correctness and validates core functionalities.
- This helps detect bugs early in the development lifecycle and improves code reliability.

4.2 CI Validation

- Git push triggers Jenkins automatically
- Build stage verifies integration integrity

4.3 Security Validation

- Trivy scans Docker images
- Detects HIGH and CRITICAL vulnerabilities

Ensures secure container deployment

4.4 Deployment Validation

- `kubectl get pods -n dev`
- `kubectl rollout status deployment/simple-node-app`

Ensures successful rollout in Kubernetes

4.5 Rollback Validation

- If production deployment fails, this command will run and it will automatically restores the previous stable version :

`kubectl rollout undo deployment/simple-node-app -n prod`

5. MONITORING & LOGGING

5.1 Logging Strategy

- **Kubernetes Logs:**
`kubectl logs -l app=simple-node-app` → Used to view application runtime logs from running pods in the cluster.
- **Jenkins Logs:**
 - Pipeline execution history
 - Build status tracking
 - Error debugging logs→ Used to monitor CI/CD pipeline execution and identify failures at each stage.

5.2 Monitoring Strategy (Recommendation)

- Prometheus : Used for collecting real-time metrics such as CPU usage, memory usage, and Kubernetes pod health.
- Grafana : Used for visualizing metrics in dashboards for better system monitoring and analysis.

To set up monitoring, install Prometheus in the Kubernetes cluster to collect real-time metrics such as CPU usage, memory usage, and pod health. After that, install Grafana and configure it to use Prometheus as a data source. Finally, import Kubernetes dashboards in Grafana to visualize cluster performance and monitor system health in real time.

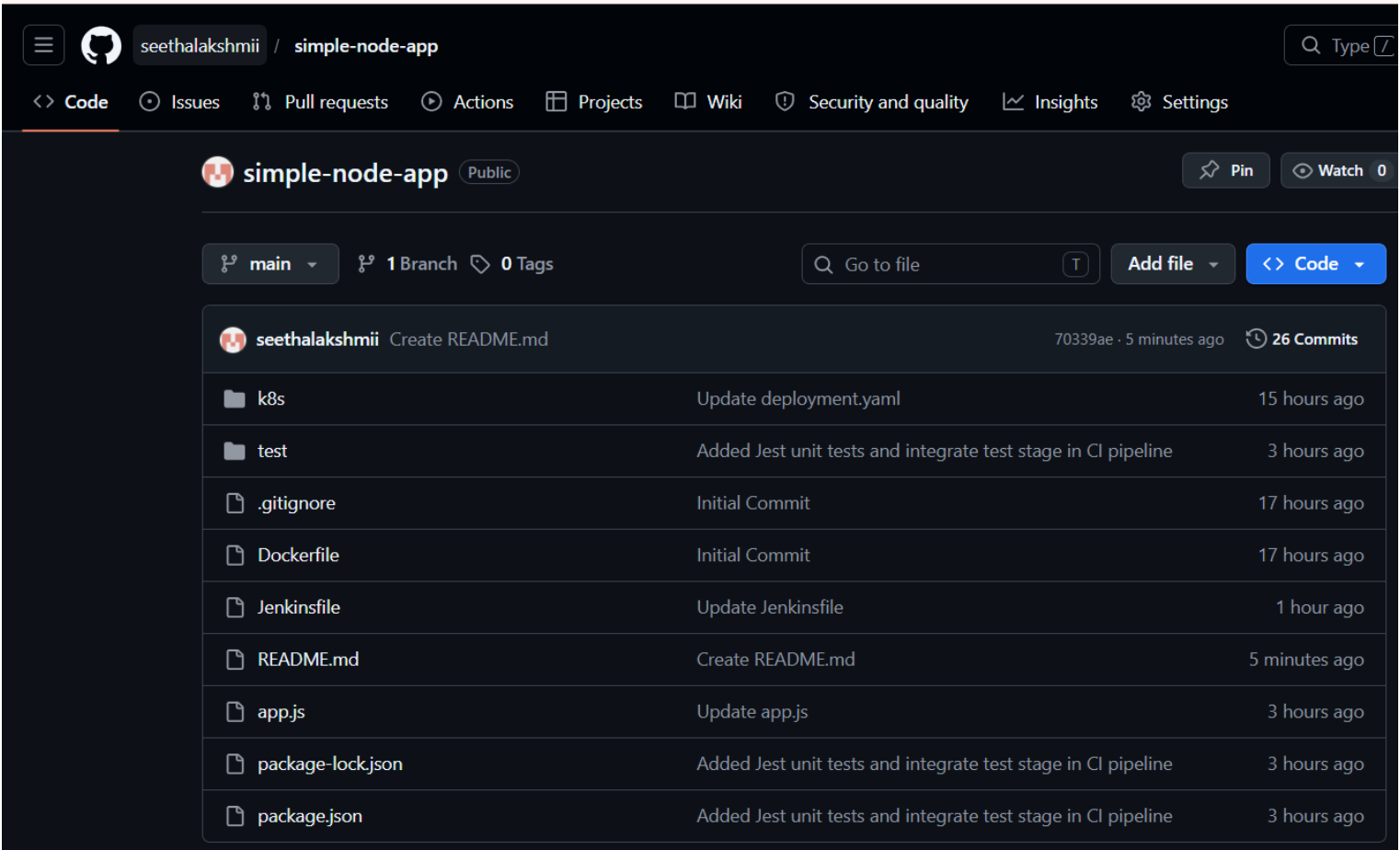
6. ACCESSING THE LIVE APPLICATION

The deployed application can be accessed after successful deployment to the Kubernetes cluster using the command :

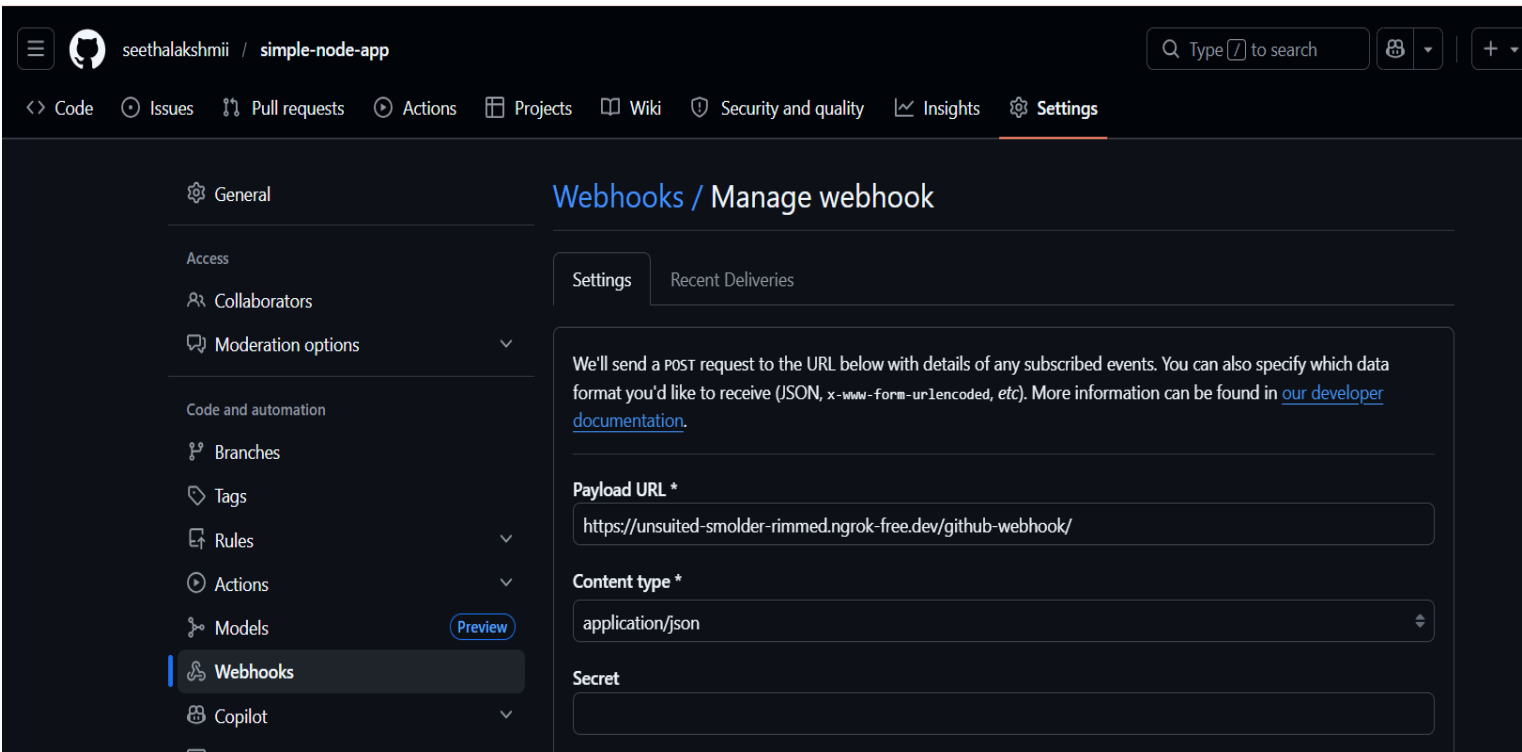
`minikube service simple-node-app -n dev`

7. SCREENSHOTS

7.1 GitHub Repository

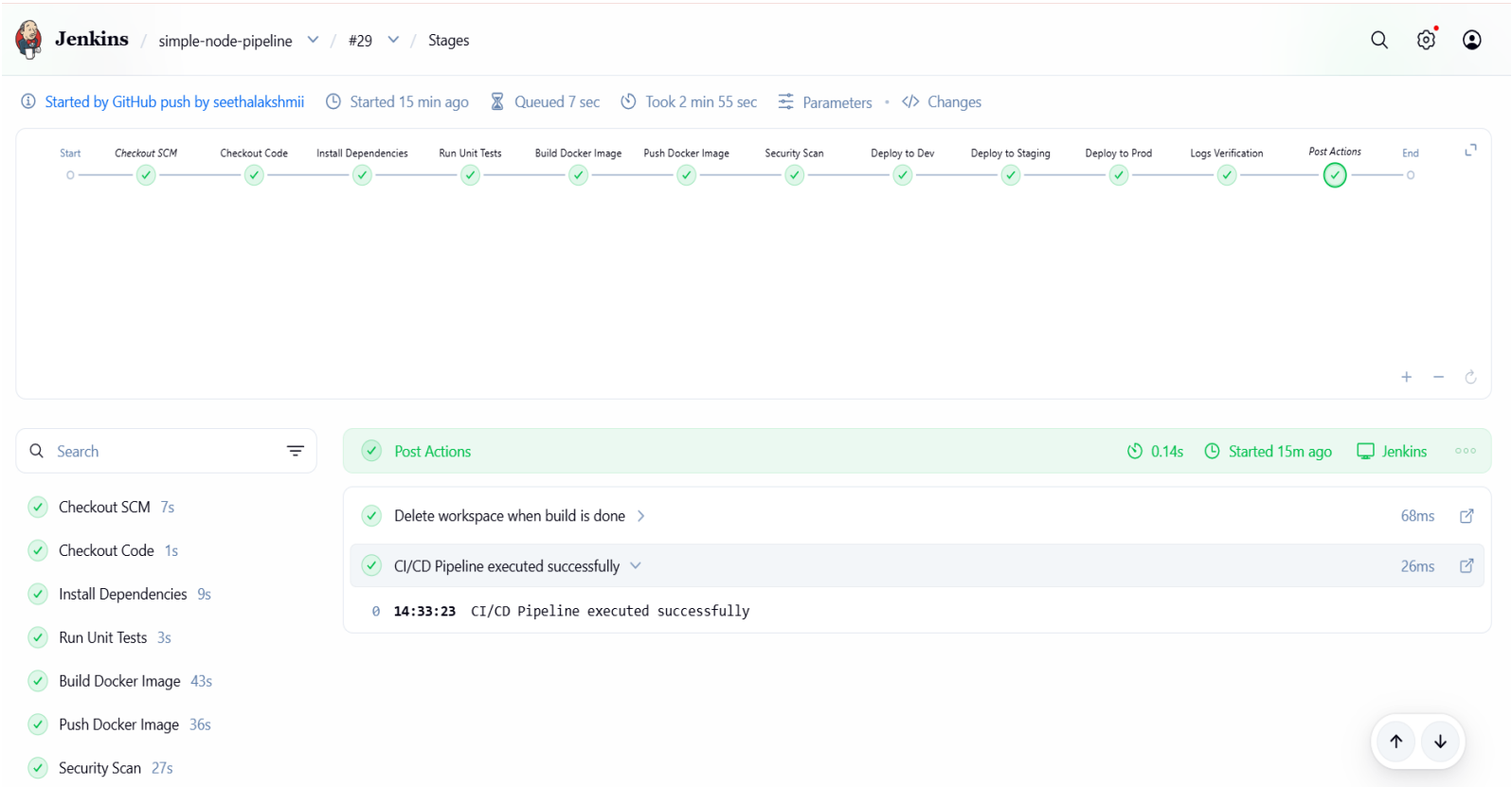


7.2 GitHub Webhook Configuration (ngrok)



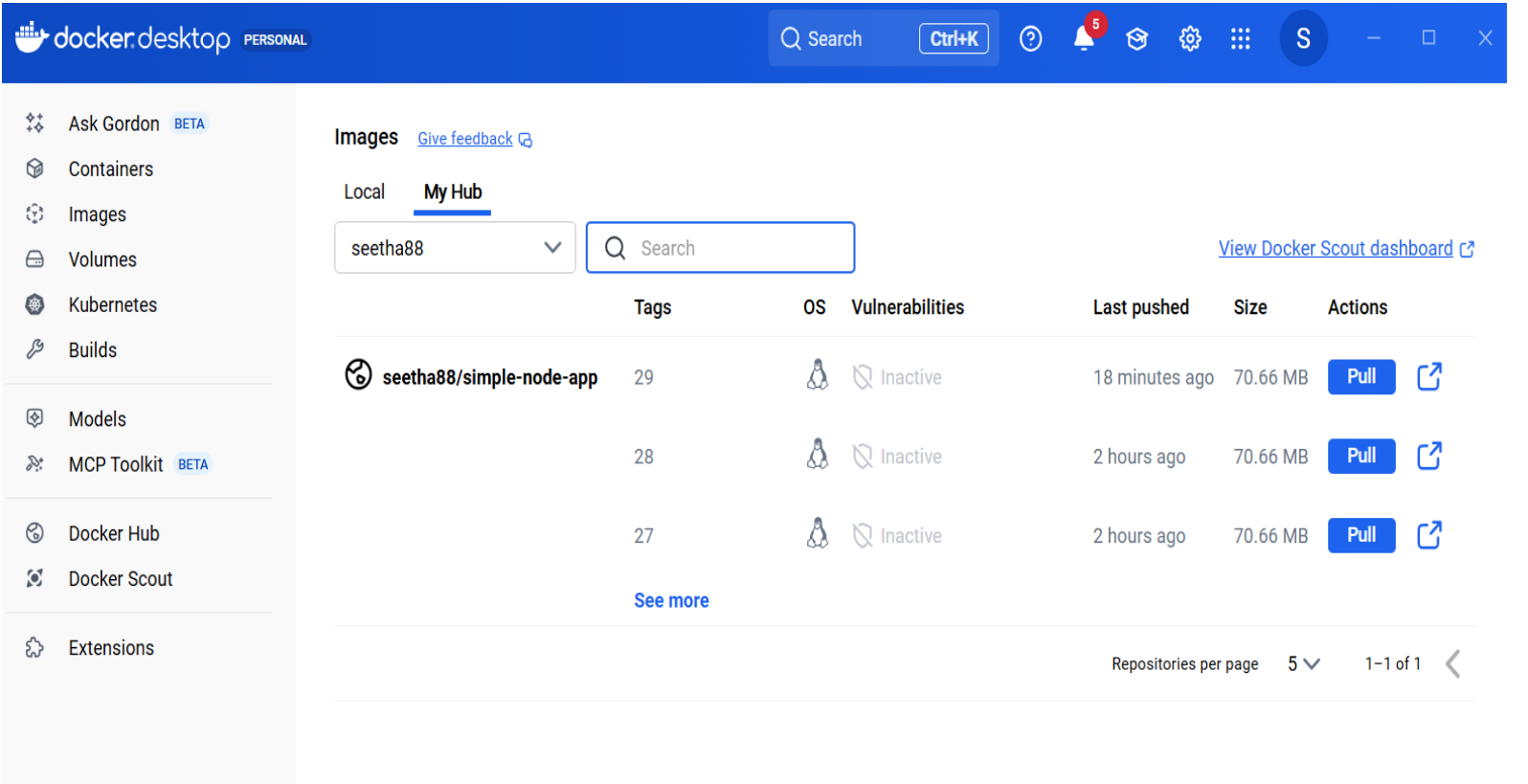
GitHub webhook configured with ngrok URL (https://unsuited-smolder-rimmed.ngrok-free.dev/github-webhook/) enabling automatic Jenkins pipeline trigger on every code push.

7.3 Jenkins Dashboard



Jenkins pipeline execution view showing all stages including checkout, build, test, security scan, and deployment.

7.4 Docker Image in DockerHub



DockerHub repository displaying successfully pushed Docker images with version tags based on Jenkins build numbers.

7.5 Kubernetes Pods Status

```
PS C:\Users\user\Downloads\simple-node-app> kubectl get pods -n dev
NAME                                READY   STATUS    RESTARTS   AGE
simple-node-app-6d67f5b74b-7k9vm    1/1     Running   0           20m
simple-node-app-6d67f5b74b-f6cv7    1/1     Running   0           20m
PS C:\Users\user\Downloads\simple-node-app>
```

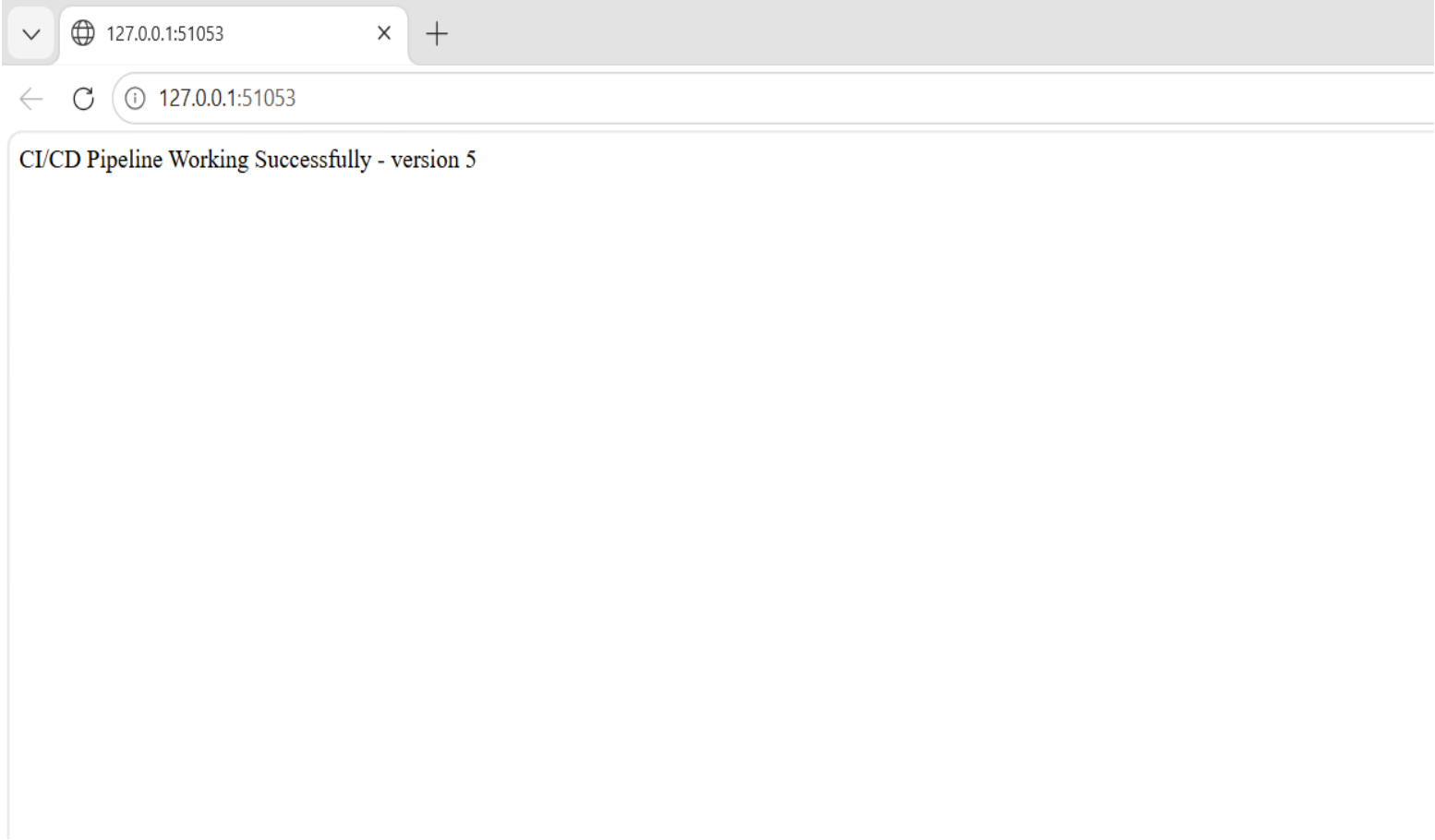
Kubernetes cluster showing running pods in the dev namespace, confirming successful deployment.

7.6 Kubernetes Services

```
PS C:\Users\user\Downloads\simple-node-app> kubectl get svc -n dev
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
simple-node-service  NodePort    10.98.206.178 <none>       80:32627/TCP     14h
PS C:\Users\user\Downloads\simple-node-app>
```

Kubernetes service output showing exposed application services for internal/external access.

7.7 Application Running in Browser



Deployed application successfully running and accessible via Kubernetes service URL.

8. Conclusion

This project demonstrates a fully automated CI/CD pipeline using GitHub, Jenkins, Docker, Kubernetes, and ngrok. The pipeline successfully automates the entire software delivery process, including code integration, build, testing, security scanning, containerization, and deployment to Kubernetes environments.

The system ensures continuous integration through automatic Jenkins triggers on every GitHub commit and continuous deployment across different namespaces. It also integrates security scanning using Trivy and supports rollback functionality to maintain application stability in case of failures.

Overall, this implementation delivers a scalable, reliable, and efficient DevOps workflow that reduces manual effort, improves deployment consistency, and enhances the speed and quality of software delivery.
